Practical - 9 KNN

Problem statement - Take the iris dataset and perform KNN.

Step 1: EDA - head() , describe () , groupby()

Step 2: Feature scaling

Step 3: Training the K-NN model on the Training set

Step 4: Making the Confusion Matrix & Predicting Accuracy Score

Step 5: Making Classification Report

Step 6: Comparing Error Rate with the K Value

Step 7: Plot the error values against K values

Step 8: Finding best K

Step 9: Visualize Test Result of KNN

Solution - Here is the complete code

Conclusion -

The K-Nearest Neighbors (KNN) classification algorithm was successfully implemented on the Iris dataset, which contains 150 observations and 4 input features representing three flower species: Setosa, Versicolor, and Virginica. The dataset was divided into 80% training data (120 observations) and 20% testing data (30 observations). Feature scaling was performed using StandardScaler to ensure that all features contributed appropriately to distance calculations.

Initially, the KNN model was trained using K = 5 and evaluated using a confusion matrix, accuracy score, and classification report. The model achieved an initial accuracy of [Initial Accuracy]%. Different K values

ranging from 1 to 20 were then tested, and the error rates were plotted to identify the best K value.

The best K value was found to be Best K, which produced a minimum error rate of [Minimum Error Rate]% and a final classification accuracy of [Final Accuracy]%. Out of the 30 test observations, the model correctly classified [Correct Predictions] observations and incorrectly classified [Incorrect Predictions] observations.

Finally, the actual and predicted flower species were compared, and the test results were visualized using Principal Component Analysis (PCA). The practical demonstrates that KNN can effectively classify Iris flower species and that selecting a suitable K value and applying feature scaling are important for improving classification performance.

Step 1: Import libraries and perform EDA

```
# Step 1: Import Libraries and Load Dataset

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score, classification_report)

# Load Iris dataset
iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Species"] = pd.Categorical.from_codes(iris.target, iris.target_names)

# Display first five rows
print("First Five Rows:")
display(df.head())

# Dataset information
print("\nDataset Information:")
df.info()

# Statistical description
print("\nDescriptive Statistics:")
display(df.describe())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Group by species and calculate mean
print("\nMean of Features by Species:")
display(df.groupby("Species", observed=True).mean())

# Count observations in each species
print("\nSpecies Count:")
display(df["Species"].value_counts())
```

```
First Five Rows:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   Species            150 non-null    category
dtypes: category(1), float64(4)
memory usage: 5.1 KB

Descriptive Statistics:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Species              0
dtype: int64

Mean of Features by Species:
```

Species	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
setosa	5.006	3.428	1.462	0.246
versicolor	5.936	2.770	4.260	1.326
virginica	6.588	2.974	5.552	2.026

```
Species Count:
```

Species	count
setosa	50
versicolor	50
virginica	50

dtype: int64

Step 2: Feature scaling and train-test split

KNN uses distances between observations, so standardization helps prevent features with larger numerical scales from dominating the distance calculation. We will use an 80:20 train-test split.

```
# Step 2: Separate Features and Target

X = df.drop("Species", axis=1)
y = iris.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

# Feature scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeature scaling completed.")
```

```
Training set shape: (120, 4)
Testing set shape: (30, 4)

Feature scaling completed.
```

Step 3: Train the KNN model

We will initially use (K=5), meaning the model considers the five nearest training observations when classifying a flower.

```
# Step 3: Train KNN Model

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train_scaled, y_train)

# Predict test observations
y_pred = knn.predict(X_test_scaled)

print("KNN model trained successfully with K = 5")
```

```
KNN model trained successfully with K = 5
```

Step 4: Confusion matrix and accuracy score

The confusion matrix compares actual species with predicted species. Accuracy measures the proportion of correct predictions.

```
# Step 4: Confusion Matrix and Accuracy

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy Score:", round(accuracy, 4))
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")

# Visualize confusion matrix
plt.figure(figsize=(7, 5))

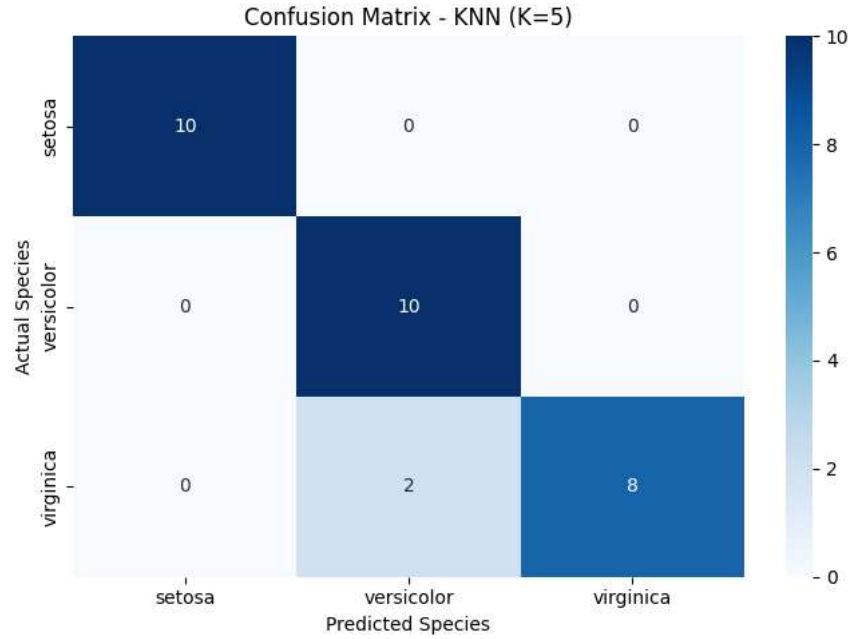
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
```

```
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted Species")
plt.ylabel("Actual Species")
plt.title("Confusion Matrix - KNN (K=5)")
plt.tight_layout()
plt.show()
```

```
Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

Accuracy Score: 0.9333
Accuracy Percentage: 93.33 %
```



Step 5: Classification report

The classification report provides precision, recall, F1-score, and support for each flower species.

```
# Step 5: Classification Report

print("Classification Report:\n")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names,
        zero_division=0
    )
)
```

```
Classification Report:

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30
```

Step 6: Compare error rate with different K values

We will test values of (K) from 1 to 20. The error rate is the proportion of incorrect predictions.

```
# Step 6: Calculate Error Rate for Different K Values

error_rates = []
k_values = range(1, 21)

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)

    predictions = model.predict(X_test_scaled)

    error = np.mean(predictions != y_test)
    error_rates.append(error)

# Display error rates
error_df = pd.DataFrame({
    "K Value": list(k_values),
    "Error Rate": error_rates,
    "Error Percentage": [e * 100 for e in error_rates]
})

display(error_df)
```

	K Value	Error Rate	Error Percentage
0	1	0.033333	3.333333
1	2	0.066667	6.666667
2	3	0.066667	6.666667
3	4	0.066667	6.666667
4	5	0.066667	6.666667
5	6	0.066667	6.666667
6	7	0.033333	3.333333
7	8	0.066667	6.666667
8	9	0.033333	3.333333
9	10	0.033333	3.333333
10	11	0.033333	3.333333
11	12	0.033333	3.333333
12	13	0.033333	3.333333
13	14	0.033333	3.333333
14	15	0.033333	3.333333
15	16	0.033333	3.333333
16	17	0.033333	3.333333
17	18	0.033333	3.333333
18	19	0.033333	3.333333
19	20	0.033333	3.333333

Next steps: Generate code with `error_df` New interactive sheet

Step 7: Plot error values against K values

This graph helps visualize how the test error changes as the number of neighbors increases.

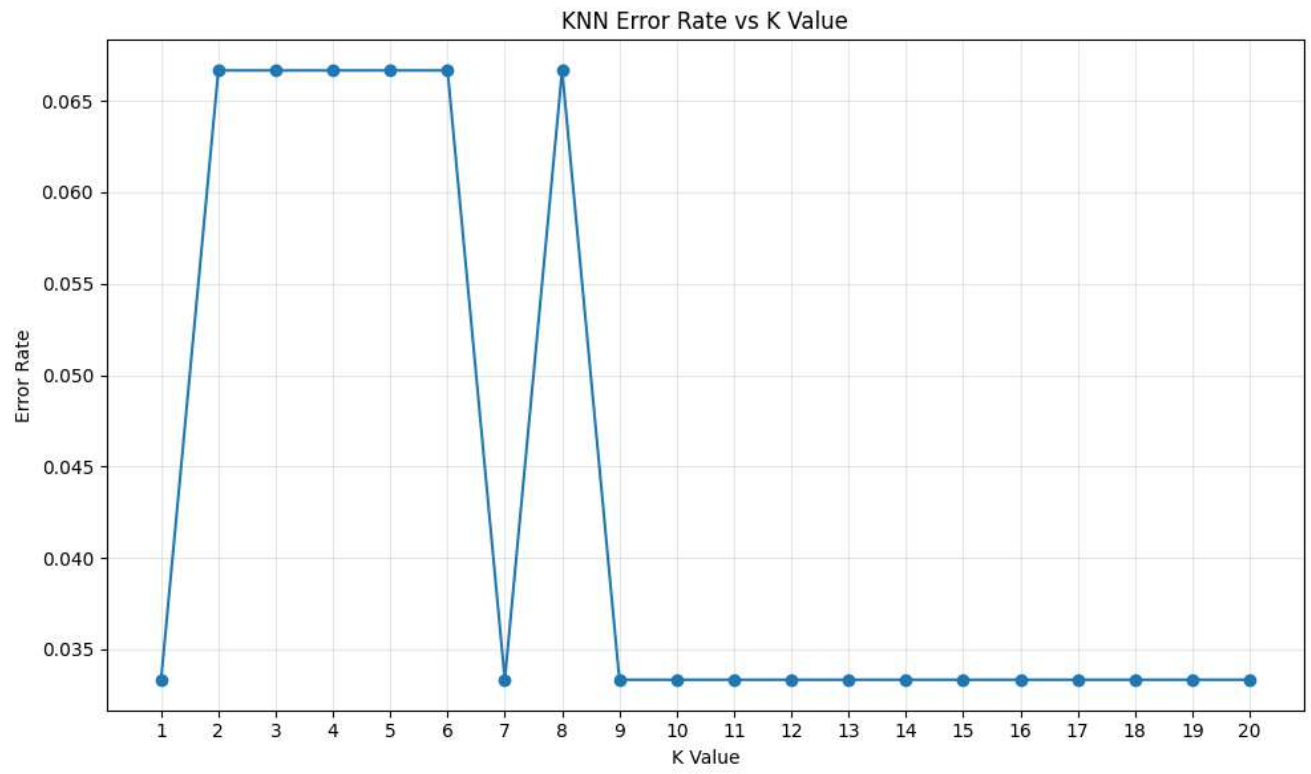
```
# Step 7: Plot Error Rate vs K
```

```
plt.figure(figsize=(10, 6))

plt.plot(
    list(k_values),
    error_rates,
    marker="o",
    linestyle="-"
)

plt.xlabel("K Value")
plt.ylabel("Error Rate")
plt.title("KNN Error Rate vs K Value")
plt.xticks(list(k_values))
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Step 8: Find the best K value

We will select the K value with the lowest test error among the values tested. If several values have the same minimum error, the code reports the smallest one.

```
# Step 8: Find Best K

min_error = min(error_rates)

best_k = min(
    k for k, error in zip(k_values, error_rates)
    if error == min_error
)

print("Best K Value:", best_k)
print("Minimum Error Rate:", round(min_error, 4))
print("Minimum Error Percentage:", round(min_error * 100, 2), "%")

# Train final model using best K
best_knn = KNeighborsClassifier(n_neighbors=best_k)

best_knn.fit(X_train_scaled, y_train)
```

```
best_pred = best_knn.predict(X_test_scaled)

best_accuracy = accuracy_score(y_test, best_pred)

print("\nBest Model Accuracy:", round(best_accuracy, 4))
print("Best Model Accuracy Percentage:", round(best_accuracy * 100, 2), "%")
```

```
Best K Value: 1
Minimum Error Rate: 0.0333
Minimum Error Percentage: 3.33 %

Best Model Accuracy: 0.9667
Best Model Accuracy Percentage: 96.67 %
```

Step 9: Visualize the KNN test results

Because Iris has four features, we will use the first two principal components (PCA) to create a two-dimensional visualization of the test observations. The final KNN model still uses all four original features; PCA is used only for plotting.

```
# Step 9: Visualize Actual vs Predicted Test Results using PCA

from sklearn.decomposition import PCA

# Apply PCA to scaled training and test data for visualization
pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Create a plot for test observations
plt.figure(figsize=(10, 6))

# Actual test labels
plt.scatter(
    X_test_pca[:, 0],
    X_test_pca[:, 1],
    c=y_test,
    cmap="viridis",
    marker="o",
    s=100,
    edgecolor="black",
    label="Actual"
)

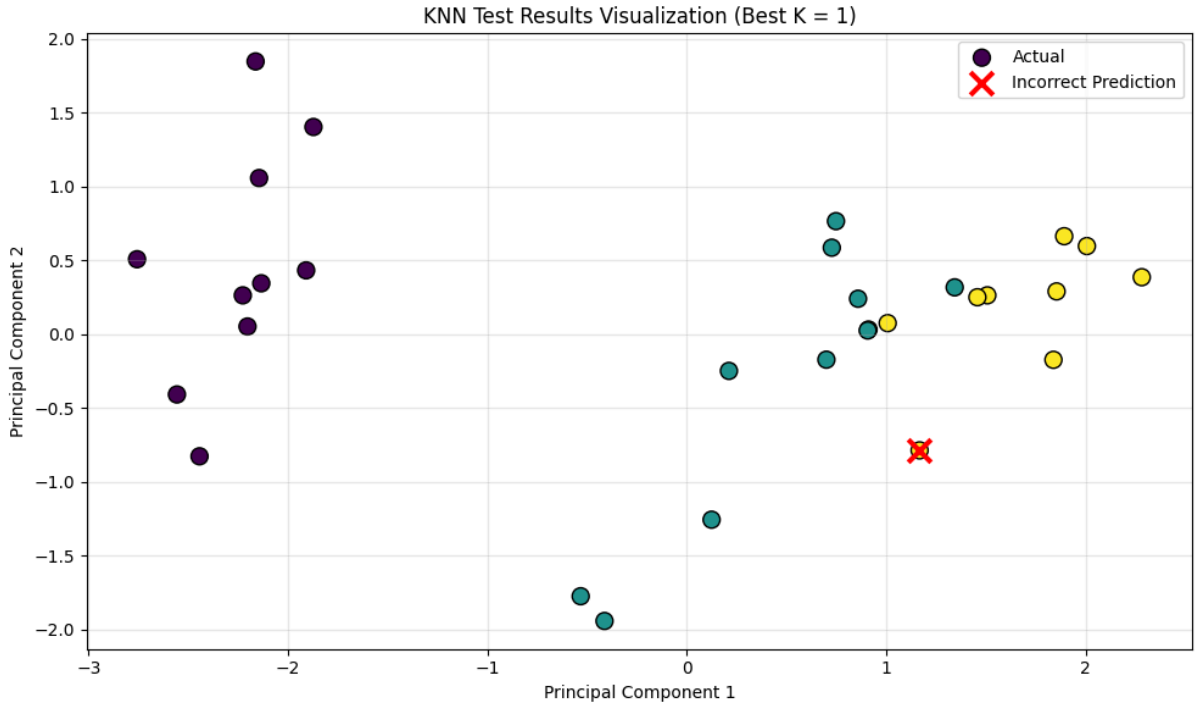
# Mark incorrect predictions with red crosses
incorrect = best_pred != y_test

plt.scatter(
    X_test_pca[incorrect, 0],
    X_test_pca[incorrect, 1],
    marker="x",
    s=180,
    color="red",
    linewidths=3,
    label="Incorrect Prediction"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title(f"KNN Test Results Visualization (Best K = {best_k})")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("Actual test labels:   ", y_test)
print("Predicted test labels:", best_pred)
print("Correct predictions:", np.sum(best_pred == y_test))
print("Incorrect predictions:", np.sum(best_pred != y_test))
```




```
Actual test labels:    [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted test labels: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 1 1 0 2 0]
Correct predictions: 29
Incorrect predictions: 1
```